PROVEIT RESEARCH NOTE

# Shorter K215 proofs

## Through checked local contracts

A concrete continuation of the Rocq and Lean proof simplification work

Prepared for Vladimir Reshetnikov

October 22, 2026 (Pacific time)

**Reviewed repository snapshot**

https://github.com/VladimirReshetnikov/ProveIt/commit/cbf94cb552a3b49509e0eee80e7455d4eb5974c6

**Recommendation.** Continue the semantic-contract route. Prove the effect of a local routine once, quantify over the preserved context, and bind each use with a small finite check. In Lean, this realizes the earlier left-frame proposal. In Rocq, a closed subprogram footprint permits reuse of existing macro proofs.

**New checked work.** This delivery supplies a guarded frame theorem, positive Home composition, universal source-path framing, and literal checks for the 35 inherited local templates. It also supplies a Coq-checked transfer theorem for an exact 89-node common subprogram. These are component results. They have not replaced the production endpoint or constituted a new full K215 build.

**Archive contents:** this article and its LaTeX source, new proof modules, pinned prerequisite sources, deterministic generators, source census, compiler logs, and reproduction instructions.

**Abstract**

The current K215 proof has already absorbed extensive sharing, reflection, and import pruning. Recommending those improvements again would obscure the remaining task. At the reviewed snapshot, the maintained Lean endpoint selects 176 project modules; its latest accepted checkpoint reports 2,052,142 selected source bytes. Its 4,751 source leaves still induce 6,176 runtime branch obligations. The best next structural reduction is to reuse exact semantic contracts across occurrences, while preserving the entire tape and source path.

We develop two concrete instances. An earlier uncompiled proposal factors 1,422 control occurrences through 35 local symbolic traces and the existing Home theorem. We supply checked universal frame and path-action lemmas, closed checks of those traces against the actual Lean table, and formal counterexamples to unsafe framing. The proposed expensive execution family has 4,789 members; this is a candidate obligation count, not a measured speedup or an integrated endpoint result. Separately, two Rocq macro libraries are identical after declared namespace/program renaming. Their 58 concrete-node contracts have 53 roots in a closed 89-node subprogram on which the old and new DAGs agree. A newly checked transport theorem preserves every exit and every final register value. We distinguish fresh component results, inherited evidence, and remaining production integration work.

# Contents

**1 The target and the present baseline** **2**

**2 What has already been simplified** **2**

**3 The organizing principle: reusable contracts** **3**

**4 A guarded left-frame theorem for the actual executor** **4**

**5 Finish at Home and recover positive progress** **5**

**6 Source-path framing: keep stripping and carry inside the suffix** **6**

**7 The checked 35-template library** **7**

**8 The endpoint must accept semantic evidence** **8**

**9 Rocq: transport macros across a checked closed footprint** **9**

**10 Recommended implementation order** **11**

**11 Validation and reproduction** **11**

**12 Conclusion** **12**

**A Primary source and artifact map** **12**

**B Attribution and originality** **13**

# 1 The target and the present baseline

The target remains the accepted two-symbol, one-tape K215 machine, started on a blank two-way infinite tape, with 215 working states and a separate halt. Initialization is included. We neither change this machine nor search for a smaller state count in this report. The relevant statement is

$$\neg\,\mathrm{Halt}(K215, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Con}(\mathsf{ZFC}). \tag{1}$$

Its positive form identifies halting with the existence of a finite ZFC derivation of falsity. The maintained Lean declaration is `KEndpoint215.blank_halts_iff_zfc_refutation`; nonhalting follows by `not_congr`. The Rocq dense endpoint is `tm215_halts_iff_ZFC_inconsistent` [1, 11].

There is a foundational distinction to preserve. The repository records external proof-assistant theorems about a syntactic ZFC proof predicate. It explicitly does not supply a derivation encoded in PA's or ZFC's own proof syntax. This report does not add that internalization. Our local operational arguments are finitary and suitable for arithmetic formalization, but compiling them in Lean or Coq is not itself an internal ZFC proof of (1). No consistency assumption is used to prove the equivalence.

All reviewed sources are pinned to commit `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`. The following literal input hashes were independently recomputed:

| Input | SHA-256 |
|---|---|
| Dense table | `fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090` |
| Native table | `9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2` |
| Source sites | `17af207195179b2508bf91154cd00739374f8862207ca21b9a3cb9098aa6f5ba` |

The accepted phase-fifteen record reports a 176-module endpoint closure, a 194-provider primary check, a 71-provider compatibility check, and 199 distinct project modules across those scopes. The original full builds reported 204 Lean project modules and 237 Rocq sources. Those counts have different scopes and dates. We attribute them to the repository; we did not rerun either complete endpoint [3, 1].

# 2 What has already been simplified

The current proof already contains most of the obvious abstractions. Its history documents shared implication derivations, pointwise lifts replacing a deduction theorem, a shared enumerator argument, generic symbolic execution, macro-word composition for AX, bisimulation symmetry, normalized source correspondence, shared Home returns, semantic seed sharing, quotient syntax, transfinite recursion, and a stopped-or-synchronized dense bridge [2].

Several large historical reductions are real but are not new savings available at this snapshot. The Lean dominated-implication core and adapter have already fallen from 689,857 to 16,826 bytes, and the AX routine from 202,222 to 7,563 bytes. Normalized correspondence already checks 1,426 entries rather than 2,240. All 4,751 source leaves remain represented. We independently counted the

1,426 literal certificate leaves in the current source data; we did not rerun that large correspondence certificate.

The pending proposals are especially relevant [4, 5]:

- The residual-decoder component provides a semantic route from exact leaf ports to source simulation. Its reported component checks are not a current full endpoint build.
- A transparent subtree abbreviation could remove repeated text in site obligations. Its quoted 18,565-byte saving is for an uncompiled source-formatting proposal, not a computational reduction.
- A four-client cofinal-scheduler abstraction shares induction, but its historical aggregate saving is only 324 bytes, and the K215-only closure grows. It may improve maintenance without improving this machine's selected proof size.
- The round-thirteen left-frame proposal discovers the 35 local templates developed below, but explicitly leaves its generic Lean proof and integration uncompiled.

The immediate opportunity is to finish a small reusable proof boundary and check its finite instantiations. Another broad simulation framework would add a new layer before removing an old one.

## 3 The organizing principle: reusable contracts

At the runtime level, different source paths execute the same finite local behavior. They differ in a preserved tape prefix, a gap length, and register contents. A useful theorem quantifies over those data instead of reevaluating the local mechanism at every occurrence.

At the register-program level, two programs may contain exactly the same closed subprogram. A macro proof should depend only on its reachable nodes. It should not be repeated because a distant dispatch routine changed.

Both arguments have the form

$$\text{one semantic theorem} + \text{ a finite applicability certificate} \Longrightarrow \text{ all concrete uses.}$$

The finite certificate is still checked. Sharing does not mean discarding occurrences, joining different source paths, or accepting a generator's output as a theorem.

Keep the measurements separate:

| Measurement | Meaning |
|---|---|
| Selected source bytes | One endpoint's transitive project source closure. |
| Maintenance source bytes | Proofs, adapters, generators, and shared definitions across supported variants. |
| Expensive execution checks | Distinct finite runtime certificates needing evaluation. |
| Kernel time and memory | Actual resource use under a specified toolchain and workload. |
| Audit surface | Semantic arguments and finite data a reviewer must understand. |

Reducing one does not automatically reduce the others. New reusable code initially increases source size until production clients replace repeated work.

# 4 A guarded left-frame theorem for the actual executor

## 4.1 The representation and the precise guard

The maintained symbolic state is

$$s = (q, o, L, R),$$

where $q$ is a state label, $o$ says whether the head is on the left list, and $L, R$ are lists of symbolic segments. A segment is a literal bit, a run of $an + c$ equal bits, or a register-file box. The interpretation `conc n vs s` produces a complete list-zipper configuration. Here $n$ is an arbitrary natural gap parameter and $vs$ is an arbitrary register file satisfying the appropriate branch validity predicate [6].

The executor's `sstep` may perform a literal transition, reorient the representation, scan a run, peel a cell, or use an already justified register-box contract. Some symbolic operations represent zero concrete transitions; a scan can represent arbitrarily many. A positive symbolic step count is not a positive concrete-time witness.

For a symbolic frame $F$, define

$$A_F(q, o, L, R) = (q, o, L +\!\!+ F, R).$$

This appends a preserved tape fragment beyond the explicit local left fragment. Define

$$G(q, o, L, R) \iff \neg(o = \mathsf{true} \wedge L = []).$$

**Theorem 4.1** (Guarded exact commutation)**.** *For every machine table $M$, branch specification $b$, frame $F$, and symbolic state $s$ satisfying $G(s)$,*

$$\mathrm{sstep}_{M,b}(A_F s) = \mathrm{map}(A_F \times \mathrm{id})(\mathrm{sstep}_{M,b}(s)).$$

*The equality preserves the entire successor state, success or failure, and the progress Boolean.*

*Proof.* Split on orientation and the leading segment. Literal transitions prepend or remove a segment at the near end and commute with appending $F$ at the far end. Run scans have the same property. The two peeling functions commute with append by a direct split on the run coefficients. The box oracle inspects its state, branch and box descriptor; its left-list updates also commute with append. The only exceptional branch is orientation-left with an empty explicit left list. The guard excludes it. The supplied Lean proof handles these cases explicitly. □

The exception is essential. With $s = (q, \mathsf{true}, [], [])$ and $F = [1]$, the unframed interpreter exposes an implicit blank, whereas the framed interpreter exposes the frame's 1. The successors differ. The supplied `unguarded_frame_counterexample` proves this for arbitrary $M$ and branch specification.

## 4.2 A finite local replay

Define `runLocal` by exact fuel, checking the guard before every symbolic operation. At fuel zero it returns the current state without demanding the guard. Thus a local trace can finish precisely at a boundary where it must stop.

**Theorem 4.2** (Universal local replay)**.** *Assume the existing register-box contracts for $M$. If*

$$\mathrm{runLocal}_{M,b}(j, s) = \mathrm{some}(t),$$

*then, for every symbolic frame $F$, every $n \in \mathbb{N}$, and every $vs$ satisfying* `Valid b vs`*,*

$$\mathrm{conc}(n, vs, A_F s) \longrightarrow^*_M \mathrm{conc}(n, vs, A_F t).$$

*Proof.* Induct on $j$. The zero case is reflexive reachability. Successful successor replay supplies the guard and a successful symbolic step. Apply Theorem 4.1, then the maintained `sstep_sound`, and compose its concrete reachability with the induction hypothesis. The executor and interpretation remain unchanged. □

The conclusion deliberately permits zero concrete steps. Positive progress comes from the final Home contract, not from counting symbolic operations.

### 4.3 Why normalization needs a boundary

The maintained left normalizer may drop trailing blank segments because they represent blank exterior in a complete configuration. This is unsafe before appending a nonblank frame. Unframed left lists [] and [0] normalize alike, but appending [1] gives [1] and [0, 1], different tapes.

The supplied `normalization_frame_counterexample` checks this using the actual `sMatches`, `conc` and frame definitions. Our replay therefore uses exact symbolic equality. Only the right endpoint is simplified by the ordinary, non-trimming segment normal form. An unframed `normL` or `sMatches` result is not lifted through a later frame.

## 5 Finish at Home and recover positive progress

All 35 inherited local templates end at exactly

$$(2, \mathsf{true}, [], R').$$

Append the frame

$$F_P = \mathrm{map}(\mathsf{bit}, \mathrm{rev}(P)) +\!\!+ [\mathsf{bit}(1)].$$

Its last bit is the existing marker. The boundary now exposes the entire preserved prefix, in reverse tape order, followed by the marker: precisely the entry needed by the maintained Home theorem [7].

The prefix premise is the existing finite scan check

$$\mathrm{scanBound}(0, \mathrm{map}(\mathsf{bit}, \mathrm{rev}(P))) = \mathrm{some}(a). \tag{2}$$

For a literal bit word, it excludes a run of eight zeros. It does not require $P$ to be nonempty or end in one. The Home theorem does require standard blank exterior beyond the marker; it does not promise behavior with arbitrary nonblank cells farther left.

**Theorem 5.1** (A positive complete-tape contract)**.** *Assume the existing box contracts, Home scan rows, and Home boundary theorem for $M$. If (2) holds and*

$$\mathrm{runLocal}_{M,b}(j, s) = \mathrm{some}(2, \mathsf{true}, [], R'),$$

*then, for every $n$ and every valid $vs$,*

$$\mathrm{conc}(n, vs, A_{F_P} s) \longrightarrow^+_M \mathrm{zc}(84, [1], P +\!\!+ \mathrm{flat}(n, vs, R')).$$

*Proof.* Theorem 4.2 reaches the framed Home port. The existing scan-bound soundness theorem supplies the unchanged-word scan of rev($P$). Append the marker's reset transition and invoke `home_word`. That theorem supplies positive reachability to dispatch. Compose the local reflexive-transitive run with this positive final segment. □

The supplied `local_home_sound` proves this statement. It quantifies over all admissible register values and all gaps, and preserves the whole represented tape. Reusing a local trace does not identify different complete source paths. A closed check of `KHomeReady.ready tm215` discharges the Home premises for the actual K215 table.

## 6 Source-path framing: keep stripping and carry inside the suffix

Physical execution alone is insufficient. We must identify the source instruction's exact successor. Write the complete consumed source word as $P +\!\!+ S$, with preserved prefix $P$ and literal local suffix $S$.

The source defines continuation by stripping $k$ one-bits from the right, and break by stripping $k$ zero-bits and then incrementing. These existing operations are total on finite words. Their exhausted-suffix behavior is unsuitable for a local frame rule: remaining work might consume the preserved prefix when one is present [8].

We introduce two partial checkers on reversed suffixes:

$$\begin{aligned}
\mathrm{stripWithin}(b, 0, xs) &= \mathrm{some}(xs),\\
\mathrm{stripWithin}(b, k+1, []) &= \mathrm{none},\\
\mathrm{incWithin}([]) &= \mathrm{none},\\
\mathrm{incWithin}(0 :: xs) &= \mathrm{some}(1 :: xs),\\
\mathrm{incWithin}(1 :: xs) &= \mathrm{map}(0 :: \cdot)(\mathrm{incWithin}(xs)).
\end{aligned}$$

The remaining stripping case consumes the next bit and decrements the required count exactly when the bit matches $b$.

**Lemma 6.1** (Local source arithmetic)**.** *Successful local stripping and, for break, successful local increment entail the corresponding equation for every prefix $P$:*

$$\begin{aligned}
\mathrm{continuePath}(k, P +\!\!+ S) &= P +\!\!+ E,\\
\mathrm{breakPath}(k, P +\!\!+ S) &= P +\!\!+ E.
\end{aligned}$$

*Here $E$ is the checked local result for the selected operation.*

*Proof.* Induct on the reversed explicit suffix. Successful partial stripping establishes that no work remains before the arbitrary reversed prefix is inspected. Successful partial increment finds a zero before carry leaves the explicit suffix. Both therefore commute with appending an arbitrary reversed prefix. Reversing the equalities yields the displayed source-path equations. □

The new `PathFrame.lean` proves these statements against the current `RegisterPath` definitions. Its combined `controlCheck_sound` covers both operations. Formal counterexamples show why exhausting a suffix while stripping, or allowing carry to escape an all-one suffix, invalidates the frame rule. Matching a few concrete source endpoints would not establish the universal fact.

# 7 The checked 35-template library

## 7.1 Use the actual table and one shared record

Native labels and maintained Lean labels differ:

$$301 \mapsto 102, \qquad 302 \mapsto 120, \qquad 254 \mapsto 230.$$

Native state 302 occurs in the traces. Our generator reads the inherited, already relabeled `FORMAL_TEMPLATES.json`; Lean directly evaluates `runLocal` against the actual `tm215`. Neither the Python trace nor the generator is assumed sound [5, 10].

All 35 local replay equations are kernel-checked. The specified operation counts sum to 444; these are symbolic executor operations, not concrete machine steps. The same generated record carries the source suffix, control operation, argument, and result. All 29 continuation and six break templates pass the new local arithmetic checker.

The right-output geometry is checked separately: $S$ is nonempty, $|E| \le |S|$, the start is the claimed primitive port, and the right output normalizes to the claimed result and adjusted gap. Thus the physical theorem and the source theorem use the same typed record. There is no unverified join between separate metadata lists.

## 7.2 The exact combined contract

For a template $t$, write $q$ for its entry state, $S$ for its literal suffix and $E$ for its result. The combined theorem `every_template_control` proves both the positive physical run below and the corresponding source equation of Lemma 6.1:

$$\begin{aligned}
&\mathrm{zc}\big(q,\ \mathrm{rev}(P \mathbin{+\!\!+} \mathrm{init}(S)) \mathbin{+\!\!+} [1],\\
&\qquad [\mathrm{last}(S)] \mathbin{+\!\!+} 0^{n+16} \mathbin{+\!\!+} [1] \mathbin{+\!\!+} \mathrm{sepsR}(vs)\big)\\
&\quad \longrightarrow^{+}_{\mathrm{tm215}}\\
&\mathrm{zc}\big(84, [1],\ P \mathbin{+\!\!+} E \mathbin{+\!\!+} 0^{\,n+(16+|S|-|E|)} \mathbin{+\!\!+} [1] \mathbin{+\!\!+} \mathrm{sepsR}(vs)\big).
\end{aligned}$$

The quantifiers cover every prefix $P$ accepted by (2), every natural $n$, and every valid register list $vs$. The register file and every preserved prefix bit remain unchanged.

The generic combined theorem retains the established box contracts as an explicit premise. The freshly compiled `every_template_control_closed` adapter in `ClosedControl215.lean` discharges it using the existing `KTape.boxLemmas`, just as the production proof does. Thus the final contract has no box, Home-row, boundary, or table-agreement assumption: only template membership, the prefix scan condition, an arbitrary gap, and a register list of length at least ten.

The production leaf uses gap $n + (64 - |Q|)$, where the whole source word $Q = P \mathbin{+\!\!+} S$ has length at most 48. Instantiate the template with

$$n' = n + (48 - |Q|).$$

Then $n' + 16 = n + (64 - |Q|)$. The length premise is essential because subtraction is truncated in the natural numbers. The correspondence is an exact arithmetic obligation, not a license to omit the full source word.

### 7.3 What the proposed reduction counts

The current site list contains 1,900 increments, 1,425 decrements, 98 breaks, 1,327 continuations, and one halt. Decrements require separate zero and positive outcomes, so

$$1900 + 2 \cdot 1425 + 98 + 1327 + 1 = 6176.$$

The inherited proposal assigns 1,422 control occurrences to 35 templates: 1,326 continuations and 96 breaks. Three controls remain literal: `001/CB1`, `1000111/CC3`, and `11011/CB1`.

| Execution family | Current occurrences | Proposed distinct checks |
|---|---|---|
| Increment | 1,900 | 1,900 |
| Decrement outcomes | 2,850 | 2,850 |
| Control | 1,425 | $35 + 3 = 38$ |
| Halt | 1 | 1 |
| Total | 6,176 | 4,789 |

The difference is 1,387, approximately 22.46% of the original expensive execution family. This is not a 22.46% compile-time improvement. All 6,176 outcomes still need coverage. Each template use needs exact routing, prefix/suffix, operation, gap and result bindings. Our new work proves the reusable universal components and the 35 local certificates; it does not count all remaining production obligations as recompiled.

## 8 The endpoint must accept semantic evidence

The current `KSim.Covered M p` requires an exact successful source walk and a Boolean `checkEdge` result for every path. That Boolean evaluates a particular symbolic executor with fuel 4,000. Eventual semantic reachability does not imply that this evaluator returns `true` within that fuel [9].

A full semantic-coverage refactor can expose positive nonhalting-edge contracts, designated halt-port contracts, the two decrement cases, bounds, register permutation, overflow, and spare-register extension. Exact source-walk totality and complete consumed words must remain part of coverage. The old Boolean checker can supply these contracts through soundness; template proofs supply them by composition [4].

There is also a smaller endpoint seam. The included `KEndpointPort.patch` factors the existing endpoint through a theorem whose operational input is simply

$$\mathrm{Halts}(M, T) \quad \longleftrightarrow \quad C222(F).$$

The dense-state, boot, and logical-calibration arguments stay unchanged. The old endpoint becomes a wrapper supplying

$$\texttt{hsrc} \circ (\texttt{KSim.halts_iff} \cdots)^{-1}.$$

A semantic residual simulation can supply the same port directly. This avoids proving Boolean evaluator acceptance from a semantic theorem.

The patch applies cleanly to the pinned source but is explicitly uncompiled against the endpoint closure. It adds 21 lines (81 to 102), so it is enabling interface work, not an immediate source-size reduction.

Decoder totality must not disappear in either migration. In the source, `step p c = none` can mean decoder exhaustion, as well as a literal halt. Preserve the existing completeness argument so that a stuck relevant source configuration corresponds to the designated halt leaf. Positive target progress must also remain available for reverse halting transport.

# 9 Rocq: transport macros across a checked closed footprint

## 9.1 The concrete repetition

The current `ProgLibPerm.v` has 37,951 bytes and 780 newline-counted lines. After removing nested comments and whitespace and applying declared namespace/program renaming, it is identical to the K218 `ProgLib219.v`. This is a reproducible textual census; semantic agreement on the shared footprint is established separately below.

The literal DAGs contain 467 and 487 nodes. The newer program changes old nodes 139, 141, 147 and 236 and adds nodes 467 through 486. The 58 concrete-node macro theorem blocks mention 53 distinct roots. Their closed transitive node footprint has only 89 nodes, none affected by those changes. Those blocks occupy 275 lines of the current macro library. The snapshot audit records all roots, nodes and source hashes.

## 9.2 An exact transport theorem

Use the existing `K230.RM.run` relation. It records a program, starting edge, initial register function, exit kind and final register function. Exits include normal, break, continue and halt. Preserving all these distinctions is stronger and more useful than a bare halting equivalence [12].

**Definition 9.1.** A finite node set $U$ is a *closed agreement footprint* for programs $p, q$ if, for every $u \in U$, the two programs have exactly equal child edges at $u$, and every child that is a node reference also belongs to $U$. Primitive edges need no membership condition.

**Theorem 9.2** (Exact macro transport)**.** *Suppose $U$ is a closed agreement footprint and $e$ is primitive or references a node in $U$. Then for all register functions $R, R'$ and all exits $x$,*

$$\mathrm{run}_p(e, R, x, R') \quad \longleftrightarrow \quad \mathrm{run}_q(e, R, x, R').$$

*Proof.* For one direction, induct on the given big-step derivation. Primitive constructors are identical. At a node constructor, table agreement rewrites the child edges, and closure permits the induction hypotheses. In the continue-one constructor, the loop restarts at the same node, which remains in $U$. The same exit and the identical final register function are preserved. Table agreement and closure are symmetric, giving the reverse direction. The supplied proof covers all sixteen constructors of the actual relation, without a termination or acyclicity hypothesis. □

The supplied `check_footprint` tests both table equality and closure. Its soundness theorem turns a true Boolean result into the hypotheses of Theorem 9.2. The concrete `Footprint215.v` kernel-checks the exact 89-node footprint and all 53 roots. Thus

```
Theorem macro_run_iff n R x R' :
  In n macro_roots ->
  (run dag219 (ENode n) R x R' <->
   run dagPerm (ENode n) R x R').
```

has no unproved table-agreement hypothesis. The companion `macro_contract` transports arbitrary preconditions and arbitrary postconditions on exits and registers. All old contracts at those roots are eligible for reuse, for every input register function.

### 9.3 The actual production refactor

The old and new libraries independently declare a nine-field record `St`. Textual identity does not make those types definitionally equal. Production must either move `St`, `get`, `set` and `approx` into a shared module with compatible exports, or supply explicit record conversions and prove their register observations agree.

Next make `sem` program-parametric, retain one shared mathematical macro library, and replace repeated variant bodies with transport wrappers. Preserve existing public statements through checked aliases or conversions. Keep changed dispatch, AX and root-program arguments local, and update the generating templates as well as generated files.

Avoid importing an entire old machine merely to shorten a local wrapper. A common macro module, parameterized by its finite table interface, gives a better dependency boundary. A canonical partial macro table with default behavior outside the closed footprint is another option: agreement matters only where the proof can go.

The pilot was freshly compiled with Coq 8.18.0 and OCaml 4.14.1 and recursively checked with `coqchk`. Five named assumption reports are closed under the global context. It is a portable Rocq-family component result, not a Rocq 9.1.0 production run. Adoption must check the wrappers and descendants under the repository's pinned production toolchain.

### 9.4 Further family sharing

The extension also repeats several runtime files across eight variants. The current census gives:

| Source | K215 bytes | Eight copies, bytes |
|---|---:|---:|
| `Runtime230Core.v` | 19,622 | 156,227 |
| `Interpreter230Sites.v` | 8,121 | 64,219 |
| `Sim230.v` | 12,037 | 95,393 |
| `Tape230Blocks.v` | 8,786 | 69,539 |
| `Tape230Carry.v` | 5,579 | 43,883 |

This suggests a subsequent family-level refactor. `Runtime230Core` already exposes its register excursion theorems as premises; its remaining table dependence can become explicit. `Sim230` is conceptually parametric in the program, checked sites, boot frame and initial registers. The Lean lane already demonstrates this architecture.

These figures quantify maintenance repetition. Deleting seven variants does not remove seven copies from one endpoint's import closure. No net source or time saving is claimed before the actual shared clients are compiled and measured.

## 10 Recommended implementation order

**First, adopt the small checked components.** Review the frame guard, full output equality, local source arithmetic and finite footprint checker. Their current proofs are included and freshly checked. Keep finite data generated from pinned inputs, with the generator outside the logical trust base.

**Second, attach the runtime contracts.** Use the submitted residual prefix route and the small semantic endpoint port, or the more general semantic-coverage interface. Bind each of the 1,422 shared control occurrences to an exact routed entry, suffix split and source action. Retain the three literal controls and all increment, decrement and halt outcomes. Full prefixes and final primitive bits remain present.

**Third, share the Rocq macro library.** Resolve record identity and exported APIs before replacing bodies. Use the 89-node gate for unchanged macros and retain independent proofs for changed parts. This is a focused transfer of an existing Lean technique to a place where Rocq still repeats it.

**Fourth, rebuild and measure the endpoints.** Recompile affected transitive closures and both literal-machine endpoints. Compare selected source bytes, maintained generator bytes, object sizes, compiler time, peak memory and endpoint assumptions against the same baseline. Historical timing must remain labeled historical.

The subtree abbreviation is a small independent cleanup. The cofinal-scheduler abstraction is a separate maintenance choice. Neither is added arithmetically to the template obligation count or treated as another reduction of the same source.

## 11 Validation and reproduction

The archive contains a machine-readable census, exact input hashes, proof sources, pinned prerequisites, accepted compiler logs, development failures kept separately, and reproduction drivers. The summary below describes fresh component checks only.

| Fresh check | Scope and result |
|---|---|
| Lean 4.32.0 | 17-module standalone closure: 11 unchanged prerequisites and six new modules. All compile. One inherited unused-simplifier-argument warning in `KZip`; none in the new modules. |
| Concrete Lean theorem | `every_template_control_closed`: no remaining box, Home, boundary or table-agreement premises. Standard three-axiom profile only. |
| Template certificates | 35 runtime equations, 35 exact geometry checks, 35 source-action checks; 29 continuation and six break templates. |
| Rocq-compatible pilot | Coq 8.18.0/OCaml 4.14.1: five modules compile; recursive `coqchk` exits zero; five named reports are closed. |
| Generation | Regenerated template source is byte-identical to the checked output. |
| Source census | 4,751 leaves, 6,176 outcomes, 1,426 normalized correspondence entries; 1,422 inherited control assignments. |
| Endpoint port | Patch applies cleanly; full endpoint compilation remains pending. |

The six new Lean modules total 36,384 bytes and 629 lines. This is additive component source, not an already achieved reduction of the production proof. The selected 17-module component totals 150,027 source bytes. Its build is not a benchmark against the complete endpoint.

No machine transition is weakened, no external oracle is introduced, and no `sorry`, `admit` or new axiom is used. Finite Lean equations use kernel checking; the Coq footprint equations use `vm_compute` followed by checked equality proofs and recursive `coqchk`.

The drivers use new output directories, verify prerequisite hashes, compile in dependency order, preserve compiler output and emit receipts. The generator can be rerun and compared byte-for-byte with its checked output. Reusable sources, rather than compiled objects, are delivered. Exact commands and toolchain requirements appear in the README.

One evidence distinction matters for future integration: historical receipts certify their recorded bytes. For example, the retained Rocq normalized-9.1 receipt lists 44 compiled entries, whereas the current local K215 source directory has 43 files and some source hashes differ. This does not invalidate the historical check or refute the machine theorem; it means that receipt alone cannot certify arbitrary later edits. Our new receipts bind only the actual sources and toolchains used here.

The production source tree was left unchanged. The endpoint-port patch is clearly separated as an uncompiled proposal. Component success is not used as evidence that this adapter or a complete endpoint passed.

## 12 Conclusion

The strongest next simplification makes each proof depend only on the computation it actually uses. For runtime code, that is a guarded local trace plus a proved return through the preserved prefix. For register macros, it is a closed subprogram footprint. Small finite certificates still establish that every application satisfies the theorem's conditions.

This delivery turns the earlier 35-template sketch into checked universal components and establishes a concrete 89-node transfer route for the Rocq macros. The remaining work is explicit: exact source-occurrence attachment, semantic endpoint integration, compatible macro wrappers, and production endpoint rebuilds. The literal machine and intended equivalence with syntactic ZFC consistency remain unchanged.

## A Primary source and artifact map

| Module or artifact | Role |
|---|---|
| `KEndpoint215` | Literal-machine halting and nonhalting statements. |
| `KEndpoint` | Dense/boot composition and ZF/ZFC calibration. |
| `KRuntimeCore` | Symbolic segments, interpretation, stepping and soundness. |
| `KHome` | Uniform prefix scan and positive Home return. |
| `KSim` | Current Boolean coverage and source simulation. |
| `KDecoderPrefix215` | Exact role/path entry and full leaf-port representation. |
| `RegisterPath` | Executable source path operations. |
| `KTable215` | Actual sparse-label table checked by local equations. |

| Module or artifact | Role |
|---|---|
| `LeftFrameContract` | New guarded frame theorem and positive composition. |
| `LeftFrameTemplates215` | New kernel-checked rendering of inherited 35-template data. |
| `LeftFrameGeometry215` | Exact input and output geometry for those records. |
| `PathFrame` | Universal local source-path arithmetic. |
| `PathFrameTemplates215` | Joined physical/source contracts and concrete Home check. |
| `ClosedControl215` | Final adapter discharging the register-engine premises. |
| `RunTransfer.v` | Generic closed-footprint transport. |
| `Footprint215.v` | Exact 89-node and 53-root certificate. |
| `KEndpointPort.patch` | Small semantic endpoint interface; uncompiled proposal. |

## B Attribution and originality

The machine, mathematical calibration and existing proof stack belong to the ProveIt project and its cited predecessors, including Elliot Glazer's K230 development and Jeff Ketchersid's register-machine and Turing-machine verification work. The accepted candidate's notice and the repository license are included.

The 35-template decomposition, 1,422 occurrence assignments and proposed 4,789-check family are credited to the repository's round-thirteen left-frame proposal. The semantic residual route is credited to the existing submitted component. Neither is presented as a new discovery.

The contribution here is the checked realization of the universal frame and source-action obligations, exact tape geometry and their joined contracts, literal kernel checks, explicit counterexamples, and the focused Rocq transfer theorem with its checked 89-node instance. No minimality, smaller machine, new consistency result, full endpoint speedup or production closure reduction is claimed.

## References

[1] ProveIt, *Formal correctness of the exact 215-state machine*. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md.

[2] ProveIt, *Shorter checked K215 proofs*. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/README.md.

[3] ProveIt, *Fifteenth K215 proof-simplification checkpoint*. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase15/README.md.

[4] ProveIt, *Batch 215: exact residual contracts and shared scheduler proofs*, with verification addendum. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/SetTheory/Cardinals/docs/reports/set-theory/zfc-closure-kernels/intake-215.md.

[5] ProveIt, *Round 13 runtime: factor at the first empty-left Home boundary*. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/proposals/round13-left-frame/runtime/README.md.

[6] ProveIt, `KRuntimeCore.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KRuntimeCore.lean.

[7] ProveIt, `KHome.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KHome.lean.

[8] ProveIt, `RegisterPath.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/RegisterPath.lean.

[9] ProveIt, `KSim.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KSim.lean.

[10] ProveIt, `KTable215.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KTable215.lean.

[11] ProveIt, `KEndpoint215.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean.

[12] ProveIt, `ProgLibPerm.v`. https://github.com/VladimirReshetnikov/ProveIt/blob/cbf94cb552a3b49509e0eee80e7455d4eb5974c6/Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/ProgLibPerm.v.